

Event Booking System

Members: Karim Bass, Ben Kasozi, Sig Anthony, Conner Hughes, Cheikh Diop



Business Situation

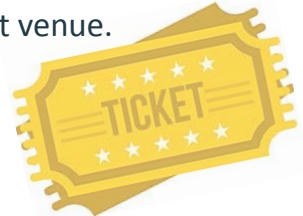


Our database is an event ticketing system which handles tickets, transactions, and events for a concert venue.

This database allows event organizers to effectively track event info, transaction statuses, and ticket inventory, and will support staff in managing operations and in adding additional events as needed.

Customers are also able to choose the event they wish to attend and purchase tickets for future events as available.

A database with these functions at minimum is central to an efficient, well-planned concert venue.





Business Functions



The database supports 3 primary business functions:

1. The database allows staff to add, modify, and change the status of events based on capacity
2. Staff and customers can view event details including its registration status and total number of seats
3. Customers using the transaction system can purchase tickets by providing their personal information and desired payment method.





Entities



Our database possesses 5 entities central to our concert venue:

Staff

Event

Customer

Ticket

Transaction





Entities



Staff

Includes unique ID, name, and role.

Event

Includes unique ID, performer name, date, total seats, and open/closed status.

Customer

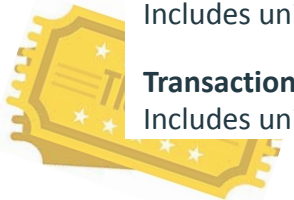
Includes unique ID and full name (first and last).

Ticket

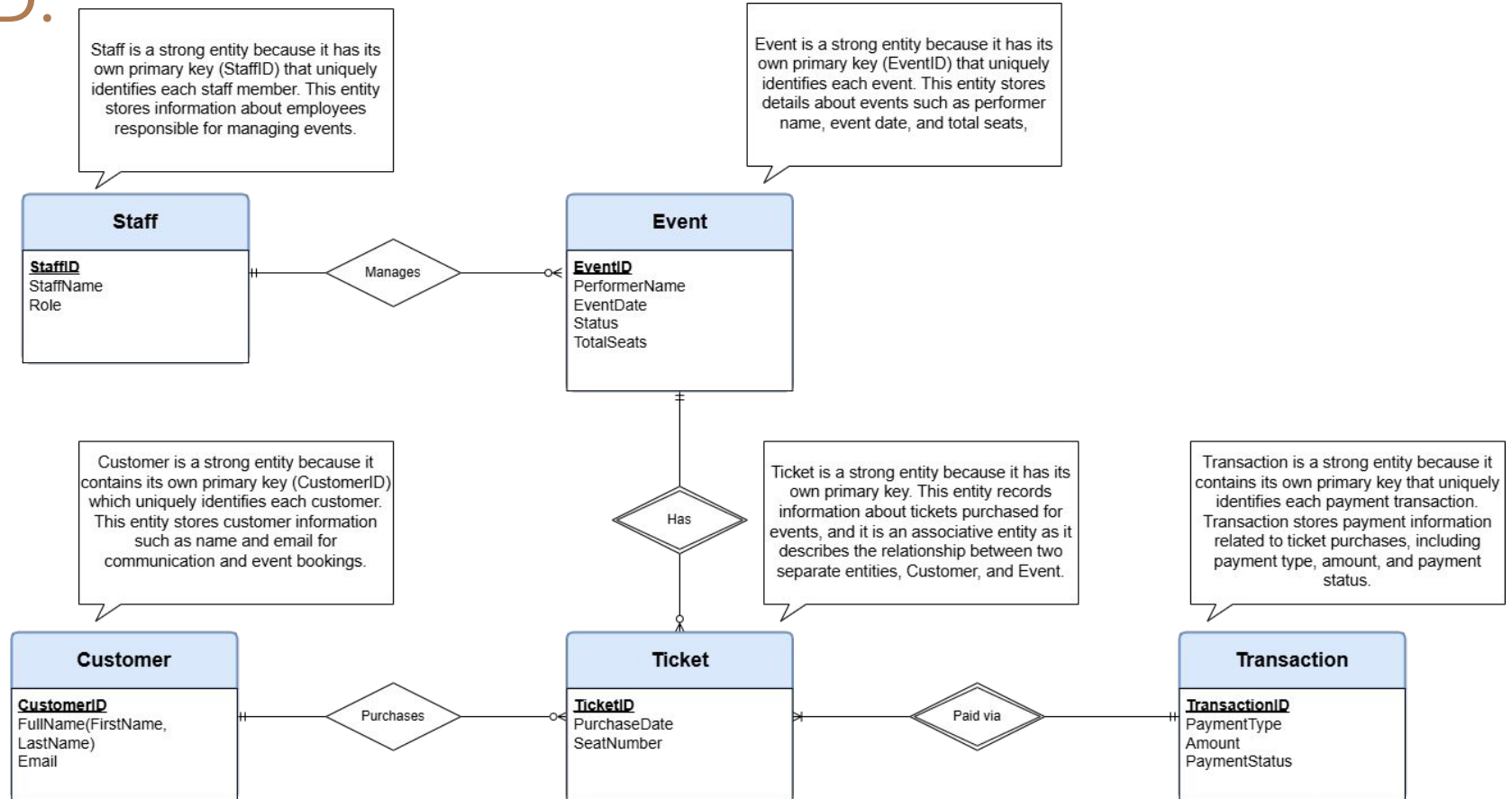
Includes unique ID, purchase date, and seat number.

Transaction

Includes unique ID, payment method, total amount, and final sales status.



ERD:





Mapped Diagram



When creating tables, the Event entity holds a foreign key, StaffID, showcasing which staff is assigned to an event.

The Ticket entity, on the other hand, possesses 3 foreign keys, EventID, CustomerID, and TransactionID. Each FK links an instance of an event, customer, and transaction, and its respective information, making Ticket both a strong entity and an associative entity.





Relational Mapping:

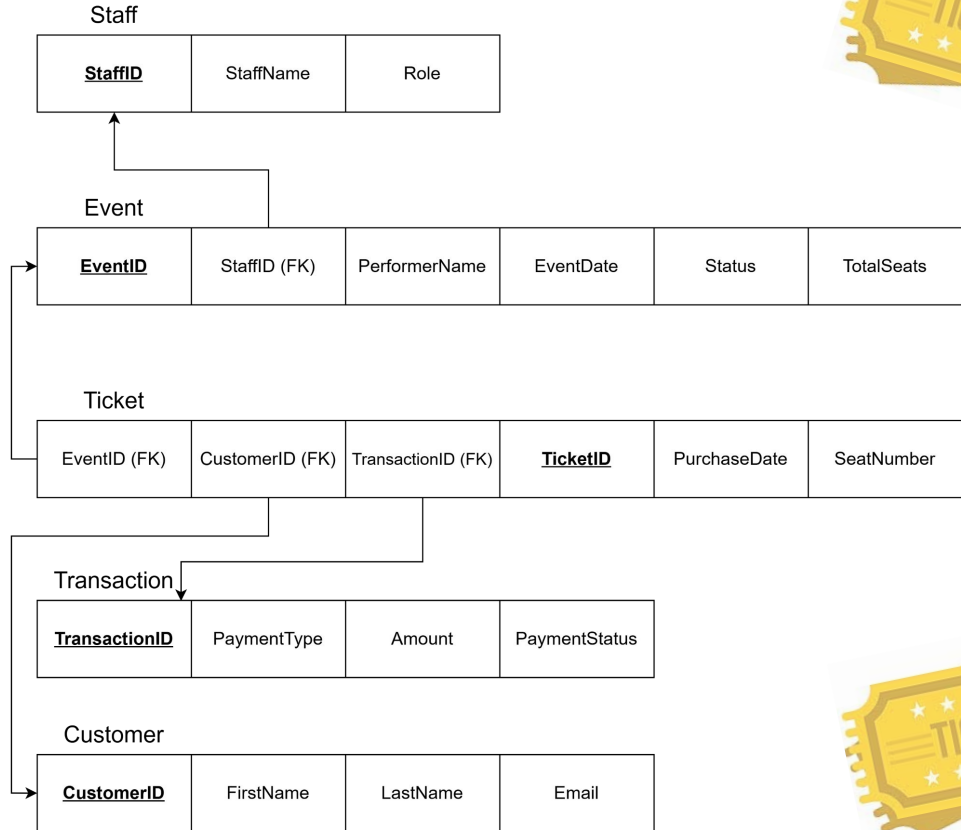


Table Syntax



- **Staff:**

```
CREATE TABLE Staff (  
    StaffID NUMBER(10) PRIMARY KEY,  
    StaffName VARCHAR2(50) NOT NULL,  
    Role VARCHAR2(50)  
);
```

- **Event:**

```
CREATE TABLE Event (  
    EventID NUMBER(10) PRIMARY KEY,  
    StaffID NUMBER(10),  
    PerformerName VARCHAR2(50) NOT NULL,  
    EventDate DATE NOT NULL,  
    Status VARCHAR2(20),  
    TotalSeats NUMBER(10),  
    CONSTRAINT fk_event_staff  
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID)  
);
```

**Customer:**

```
CREATE TABLE Customer (  
    CustomerID NUMBER(10) PRIMARY KEY,  
    FirstName VARCHAR2(50),  
    LastName VARCHAR2(50),  
    Email VARCHAR2(320) NOT NULL UNIQUE  
);
```



Table Syntax



Transaction:

```
CREATE TABLE Transaction (  
    TransactionID NUMBER(10) PRIMARY KEY,  
    PaymentType VARCHAR2(50),  
    Amount NUMBER(10,2),  
    PaymentStatus VARCHAR2(50)  
);
```



Ticket:

```
CREATE TABLE Ticket (  
    EventID NUMBER(10) NOT NULL,  
    CustomerID NUMBER(10) NOT NULL,  
    TransactionID NUMBER(10),  
    TicketID NUMBER(10) PRIMARY KEY,  
    PurchaseDate DATE,  
    SeatNumber VARCHAR2(10),  
    CONSTRAINT fk_ticket_event  
        FOREIGN KEY (EventID) REFERENCES  
            Event(EventID),  
    CONSTRAINT fk_ticket_customer  
        FOREIGN KEY (CustomerID) REFERENCES  
            Customer(CustomerID),  
    CONSTRAINT fk_ticket_transaction  
        FOREIGN KEY (TransactionID) REFERENCES  
            Transaction(TransactionID)  
);
```





Record Insertions



To Staff:

- INSERT INTO Staff VALUES (1, 'Alice Johnson', 'Manager');
- INSERT INTO Staff VALUES (2, 'Bob Smith', 'Coordinator');

To Event:

- INSERT INTO Event VALUES (101, 1, 'Drake', DATE '2024-12-01', 'Active', 5000);
- INSERT INTO Event VALUES (102, 2, 'Taylor Swift', DATE '2024-12-10', 'Active', 8000);

To Customer:

- INSERT INTO Customer VALUES (1, 'John', 'Doe', 'john@email.com');
- INSERT INTO Customer VALUES (2, 'Sarah', 'Lee', 'sarah@email.com');
- INSERT INTO Customer VALUES (3, 'Mike', 'Chen', 'mike@email.com');





Record Insertions



To Transaction:

- INSERT INTO Transaction VALUES (301, 'Credit Card', 150.00, 'Completed');
- INSERT INTO Transaction VALUES (302, 'PayPal', 200.00, 'Completed');
- INSERT INTO Transaction VALUES (303, 'Debit Card', 180.00, 'Pending');

To Ticket:

- INSERT INTO Ticket VALUES (101, 1, 301, 201, DATE '2024-11-20', 'A1');
- INSERT INTO Ticket VALUES (101, 2, 302, 202, DATE '2024-11-21', 'B2');
- INSERT INTO Ticket VALUES (102, 3, 303, 203, DATE '2024-11-22', 'C3');



Record insertions in iSQL:



Enter statements:

```
SELECT * FROM STAFF;  
SELECT * FROM EVENT;  
SELECT * FROM CUSTOMER;  
SELECT * FROM TICKET;  
SELECT * FROM TRANSACTION;
```

STAFFID	STAFFNAME	ROLE
1	Alice Johnson	Manager
2	Bob Smith	Coordinator

EVENTID	STAFFID	PERFORMERNAME	EVENTDATE	STATUS	TOTALSEATS
101	1	Drake	01-DEC-24	Active	5000
102	2	Taylor Swift	10-DEC-24	Active	8000

CUSTOMERID	FIRSTNAME	LASTNAME	EMAIL
1	John	Doe	john@email.com
2	Sarah	Lee	sarah@email.com
3	Mike	Chen	mike@email.com

EVENTID	CUSTOMERID	TRANSACTIONID	TICKETID	PURCHASED	SEATNUMBER
101	1	301	201	20-NOV-24	A1
101	2	302	202	21-NOV-24	B2
102	3	303	203	22-NOV-24	C3

TRANSACTIONID	PAYMENTTYPE	AMOUNT	PAYMENTSTATUS
301	Credit Card	150	Completed
302	PayPal	200	Completed
303	Debit Card	180	Pending



SQL Execution: Single Table w/ Aggregation

- To find the total number of seats between all performers, using only our Event table:

Enter statements:

```
SELECT SUM(TotalSeats) AS TotalSeatCount  
FROM Event;
```

Execute

Save Script

Clear Screen

Cancel

TOTALSEATCOUNT

13000





SQL Execution: JOIN Query

- To find which customer purchased which seat number using a Join statement:

Enter statements:

```
SELECT c.FirstName, c.LastName, t.SeatNumber  
FROM Customer c  
JOIN Ticket t ON c.CustomerID = t.CustomerID;
```

Execute

Save Script

Clear Screen

Cancel

FIRSTNAME	LASTNAME	SEATNUMBER
John	Doe	A1
Sarah	Lee	B2
Mike	Chen	C3





SQL Execution: Multi-JOIN Query

- To show who bought a ticket, for which event, what seat they got, and how they paid (Combines 4 tables):

Enter statements:

```
SELECT c.FirstName,  
       c.LastName,  
       e.PerformerName,  
       t.SeatNumber,  
       tr.PaymentType,  
       tr.Amount,  
       tr.PaymentStatus  
FROM Customer c  
JOIN Ticket t  
    ON c.CustomerID = t.CustomerID  
JOIN Event e  
    ON t.EventID = e.EventID  
JOIN Transaction tr  
    ON t.TransactionID = tr.TransactionID;
```

Execute

Save Script

Clear Screen

Cancel

FIRSTNAME	LASTNAME	PERFORMERNAME	SEATNUMBER	PAYMENTTYPE	AMOUNT	PAYMENTSTATUS
John	Doe	Drake	A1	Credit Card	150	Completed
Sarah	Lee	Drake	B2	PayPal	200	Completed
Mike	Chen	Taylor Swift	C3	Debit Card	180	Pending





SQL Execution: Simple Update Query

- To update the payment status of a transaction:

Enter statements:

```
Update Transaction
SET PaymentStatus = 'Completed'
WHERE TransactionID = 303;

SELECT * FROM Transaction;
```

Execute

Save Script

Clear Screen

Cancel

1 row updated.

TRANSACTIONID	PAYMENTTYPE	AMOUNT	PAYMENTSTATUS
301	Credit Card	150	Completed
302	PayPal	200	Completed
303	Debit Card	180	Completed





SQL Execution: Update Query

```
UPDATE Event  
SET Status = 'Sold Out'  
WHERE EventID = 101;  
  
select * from event;
```

1 row updated.

EVENTID	STAFFID	PERFORMERNAME	EVENTDATE	STATUS	TOTALSEATS
101	1	Drake	01-DEC-24	Sold Out	5000
102	2	Taylor Swift	10-DEC-24	Active	8000





Questions & Answers

We welcome your feedback and inquiries.

